Module 02 · 집계 기초 (신규)

로그를 '세는' 기술 — 집계로 침해를 찾는다

검색(필터)은 '보여줘', 집계는 '세어줘 · 묶어줘'.

240만 건을 눈으로 볼 수는 없다. 집계가 수백만 줄을 한 장의 표로 접는다.

kt66 퍼플 집계 트랙 · 실제 적재 데이터 (purple-r01) 로 설명

왜 집계인가 — 검색만으로는 못 푸는 질문

Discover 의 검색 (DQL) 은 '조건에 맞는 로그를 보여줘' 다. 하지만 침해 조사의 진짜 질문은 '세고 · 묶고 · 비교' 해야 답이 나온다.

검색 (필터) 로 되는 것	집계라야 되는 것
data.id:404 인 로그 보기	404 를 가장 많이 받은 IP 는 누구인가
한 IP 의 요청 보기	어느 IP 가 가장 넓게 (고유 URL 수) 훑었나
어제 로그 보기	90 일 중 어느 시간대에 몰렸나
403 로그 보기	같은 IP 의 403 와 200 비율은

집계 (aggregation) = 수백만 건을 '버킷 (묶음)' 으로 나누고 각 버킷을 '지표 (숫자)' 로 재는 것. 이 묶음 · 재기의 조합이 13 개 라운드 탐지의 거의 전부다.

질문 여섯 개 = 집계 여섯 개

13 개 라운드의 거의 모든 탐지는 아래 여섯 조합의 변형이다 . 이 표가 이 과목의 지도다 .

질문	집계	무엇을 하나
누가 가장 많이 ?	Terms	값 (IP· 계정 ·URL) 별로 묶어 건수를 센다
얼마나 넓게 훑었나 ?	Cardinality	서로 다른 값이 몇 개인지 센다 (고유 수)
언제 , 얼마나 규칙적으로 ?	Date Histogram	시간을 일정 간격 (분 · 시간 · 일) 으로 묶는다
막힌 것 vs 통과한 것 ?	Filters	403 / 200 / 500 을 한 줄에 나눠 센다
얼마나 크게 ?	Sum · Max · Avg	바이트 · 응답 크기 같은 숫자를 잰다
조건을 넘는 것만 ?	bucket_selector	기준을 넘는 묶음만 남긴다 → 모니터 (경보)

환경

시작하기 전에 — 무엇을 어디서 하나

집계는 전부 ' 인덱서에 쌓인 로그 ' 를 대상으로 한다 . 먼저 등장인물부터 익히자 .

kt66	실습용 가상 회사 망 (웹 · 메일 · 서버 ·PC) 과 보안 관제 (SIEM) 가 한 통에 들어 있는 환경
Wazuh · 인덱서	로그를 모아 저장 · 탐지하는 SIEM. 그 안의 인덱서 (OpenSearch) 가 로그를 담는 ' 창고 '
대시보드	웹 화면 https://<kt66 호스트 >:5601 — 여기서 본다 (계정은 강사 안내)
Dev Tools	창고에 질문 (쿼리) 을 글로 직접 던지는 콘솔 — 이 과목의 주 도구
Discover · Visualize	같은 창고를 ' 클릭 ' 으로 검색 · 그래프로 보는 화면

인덱스란 ? — 로그를 담는 '서랍'(그림으로)

인덱스 (index) = purple-r01

로그(문서)를 담는 '이름표 붙은 큰 서랍'

- 문서 1 · 로그 한 줄 (사건 하나, JSON)
- 문서 2 · 로그 한 줄 (사건 하나, JSON)
- 문서 3 · 로그 한 줄 (사건 하나, JSON)
- 문서 4 · 로그 한 줄 (사건 하나, JSON)
- 문서 5 · 로그 한 줄 (사건 하나, JSON)

펼치면 →

문서 하나 = 필드(칸)들

`timestamp`	2026-10-30 04:11 (언제)
`agent.name`	web (어느 센서)
`data.srcip`	192.168.0.202 (출발지 IP)
`data.id`	404 (응답 코드)
`data.vhost`	neobank.kt66.lab (어느 사이트)
`rule.id`	31103 (탐지 룰)

쉽게 : 인덱스 = 엑셀 파일, 문서 = 행(가로줄), 필드 = 열(세로 칸). 단, 수백만 행을 순식간에 세도록 특수하게 저장한다.

인덱스를 왜 쓰나 · 실무에서는 어떻게

왜

- 로그가 수천만 건이면 파일로는 못 센다 . 인덱스는 ' 어떤 값이 어디 있는지 ' 미리 색인해 둬서 검색 · 집계가 순식간이다 .

매핑 (mapping) — 필드의 ' 종류 '

- 각 필드는 종류가 있다 : 문자열 (keyword·text) · 숫자 · 날짜 . 종류에 따라 집계 방식이 달라진다 (문자열은 묶기 , 숫자는 합 · 평균).

실무 — 시간 기반 인덱스

- Wazuh 는 하루치씩 나눠 담는다 : wazuh-alerts-4.x-2026.11.01 처럼 . 오래된 서랍은 지우고 (보존기간), 이름 패턴 wazuh-alerts-* 로 여러 날을 한꺼번에 검색한다 .

이 실습은 라운드마다 인덱스 하나 (purple-r01 … purple-r15) 에 90 일치를 담는다 .

★ 이 인덱스는 '직접 만든다' — 안 만들면 에러

purple-r01 인덱스는 처음엔 없다. 그냥 쿼리하면 "index_not_found_exception" 에러가 난다. 먼저 데이터를 만들어 넣어야 한다.

kt66 호스트에 접속 (ssh) 해서 ~/purple-agg 에서 아래를 순서대로 실행한다:

1 연결 확인

```
python3 r01_baseline.py check
```

2 적재 (수 분)

```
python3 r01_baseline.py gen --anchor 2026-11-01   ← 이때 purple-r01 이 생긴다
```

3 건수 확인

```
python3 r01_baseline.py ask Q0
```

기준일을 2026-11-01 로 고정해야 숫자가 정답 키와 같다. 다른 라운드는 r02_recon.py … 처럼 파일만 바꾼다.

데이터 만들기 — check 실행

kt66 호스트 — ~/purple-agg

```
# 1) 인덱서에 연결되는지 확인 (아직 데이터는 없다)
ccc@kt66:~/purple-agg$      python3 r01_baseline.py check
[OK] opensearch 7.10.2
health status index uuid pri rep docs.count ...
(purple-* 인덱스 없음)
```

1) 인덱서 연결 확인 — 아직 purple-* 인덱스가 없다

데이터 만들기 — gen 실행

kt66 호스트 — ~/purple-agg

```
# 2) 90일치 로그를 만들어 인덱서에 적재 — 이때 purple-r01 이 생긴다
ccc@kt66:~/purple-agg$      python3 r01_baseline.py gen --anchor 2026-11-01
   · 외부 IP 풀 생성 중 (신규 방문자 1,000,000개)…
  적재 중… 2,444,723건 (155초)
[적재] purple-r01: 2,444,723건
정상(배경 포함) 비율 99.91%  ·  소요 155초
대시보드: 인덱스 패턴 purple-* (시간 필드 timestamp), 시간 범위 Last 91 days
```

2) 생성 · 적재 — 90 일치 240 만 건을 넣으면 purple-r01 인덱스가 생긴다 (수 분)

데이터 만들기 — askq0 실행

kt66 호스트 — ~/purple-agg

```
# 3) 적재 건수 확인 — 총 이벤트 수 ¦ 서로 다른 외부 출발지 IP 수
ccc@kt66:~/purple-agg$      python3 r01_baseline.py ask Q0
# Q0 총 이벤트 수와 서로 다른 외부 출발지 IP 수
  2444723 ¦ 1049880
```

3) 적재 확인 — 총 2,444,723 건 · 서로 다른 외부 IP 1,049,880 개

인덱스 패턴 만들기 — 대시보드가 인덱스를 '보게'

Dev Tools 는 인덱스 이름 (purple-r01) 으로 바로 질의한다 . 하지만 Discover·Visualize 는 '인덱스 패턴' 이 있어야 그 서랍을 본다 .

인덱스 패턴

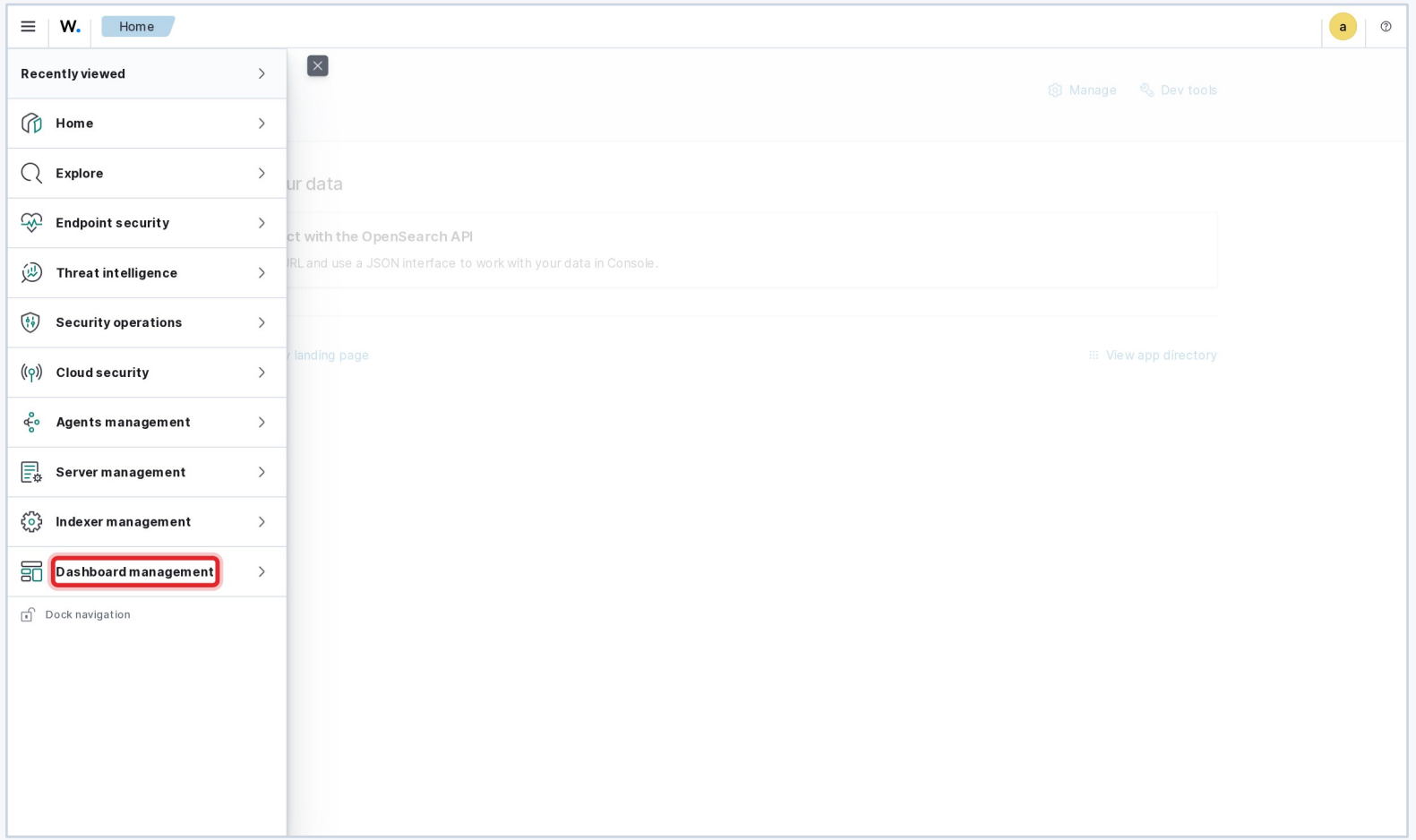
'어떤 서랍들을 볼지' 정하는 별명 . purple-* 로 만들면 purple-r01~r15 를 다 본다

만들기 (처음 한 번만)

- ☰ 메뉴 → Dashboards Management → Index patterns → Create index pattern
- 이름에 purple-* 입력 → Next → 시간 필드 (Time field) 에 timestamp 선택 → Create

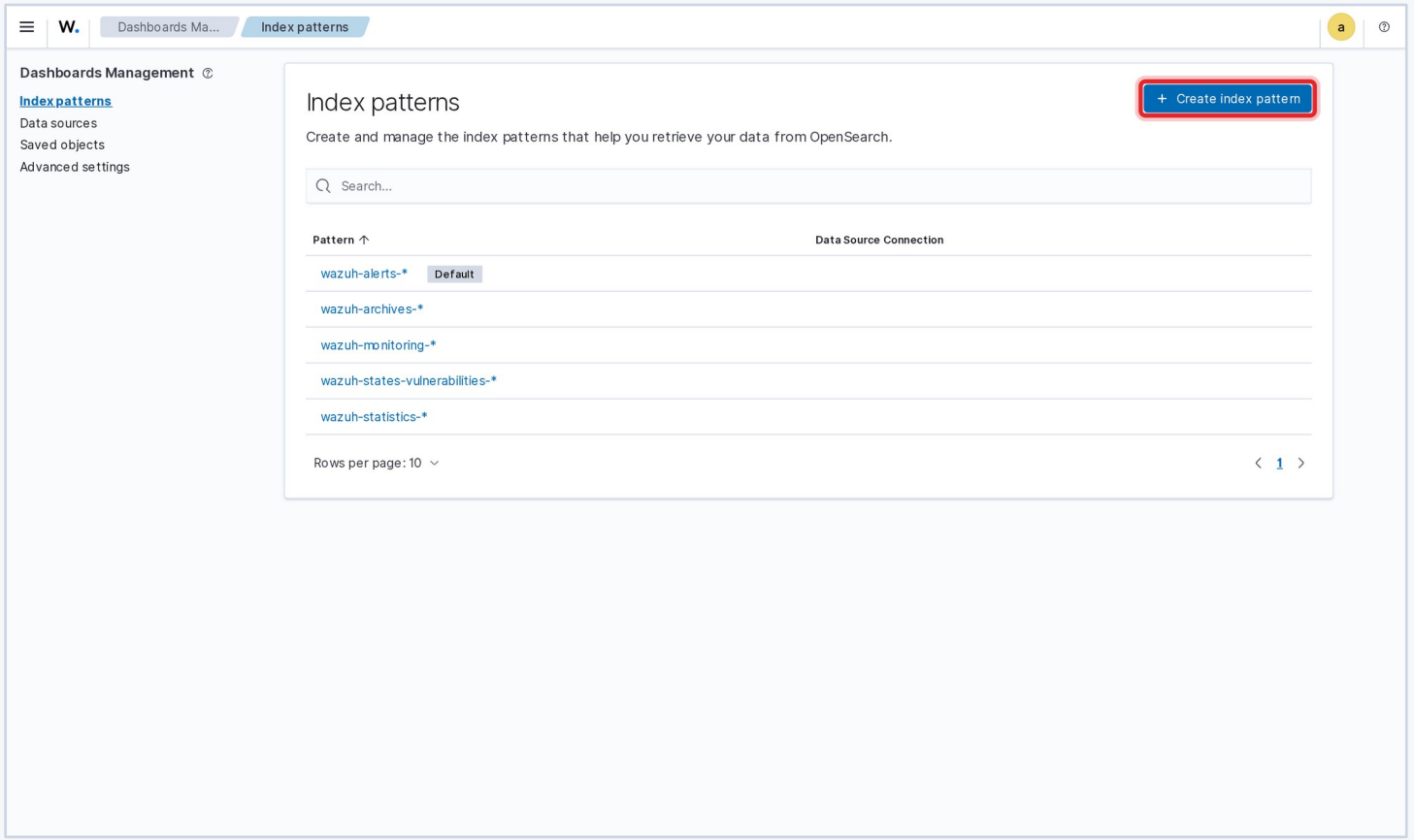
이 패턴 하나면 모든 라운드 (purple-r01~r15) 를 Discover·Visualize 에서 볼 수 있다 .

인덱스 패턴 만들기 — ① ☰ 메뉴 → Dashboard management



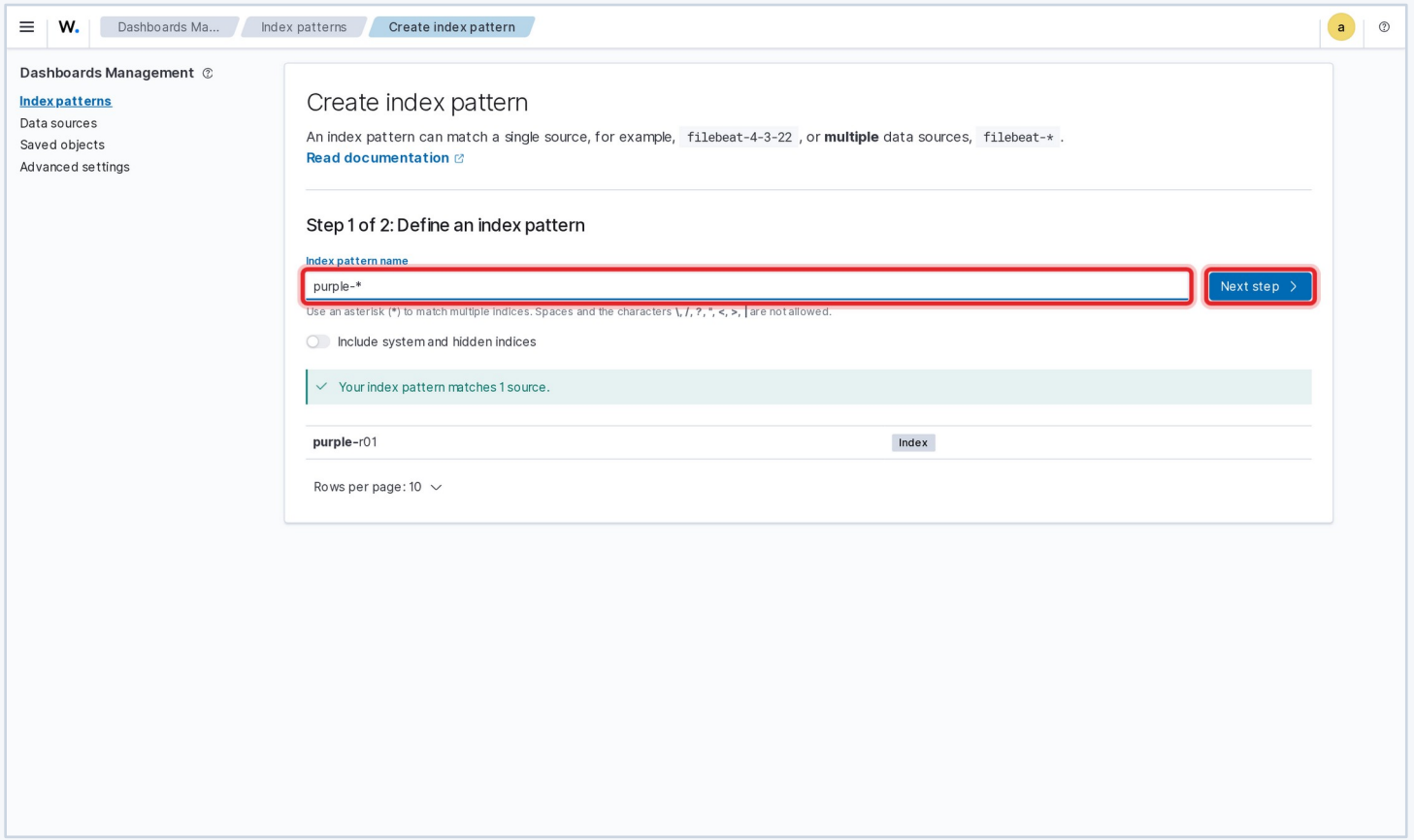
왼쪽 위 ☰ 를 열고 맨 아래 Dashboard management 를 누른다

인덱스 패턴 만들기 — ② Index patterns → Create



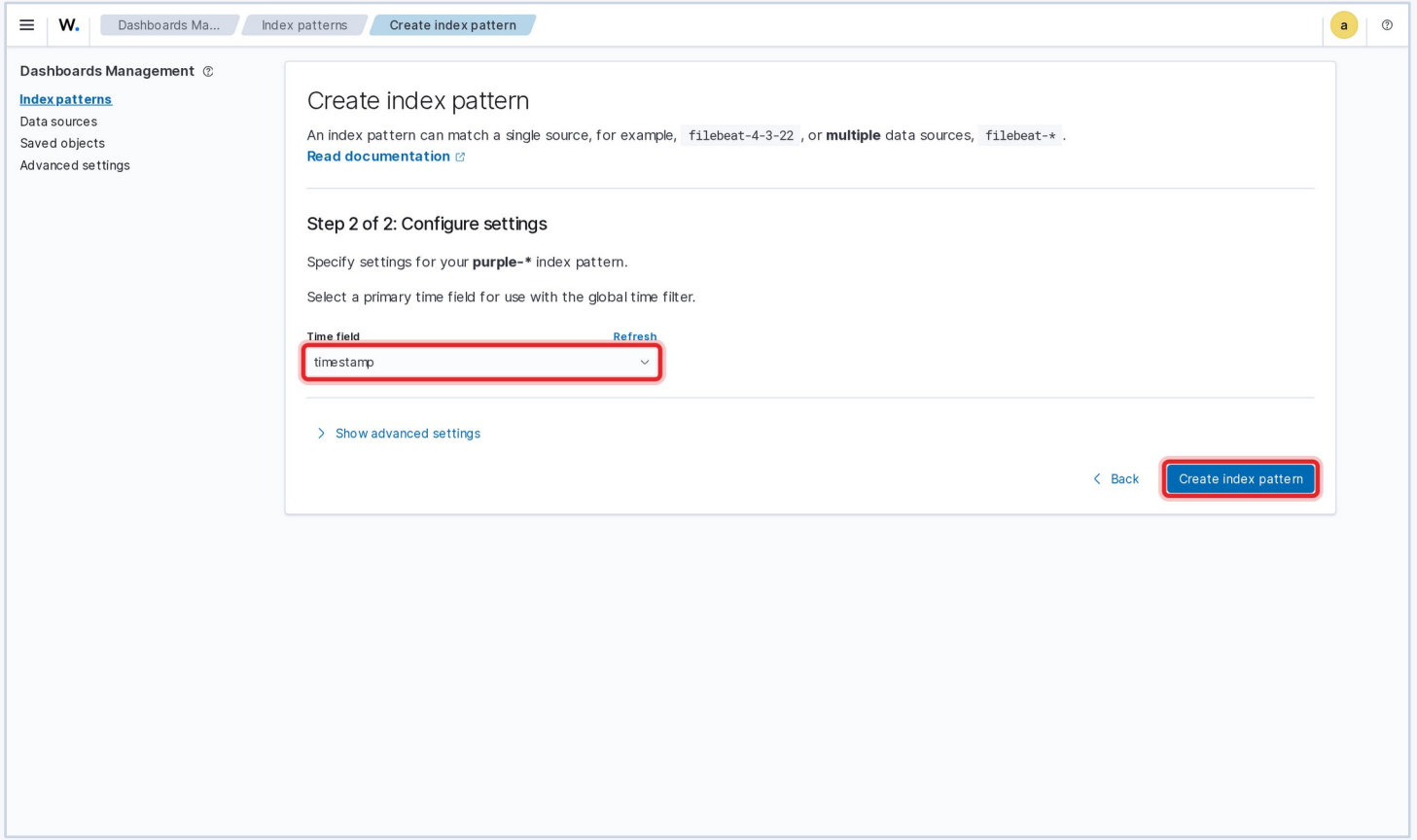
Index patterns 화면에서 오른쪽 위 Create index pattern 을 누른다 (wazuh-* 들은 kt66 기본 패턴)

인덱스 패턴 만들기 — ③ 이름에 purple-* 입력



이름에 purple-* 입력 → 'matches 1 source: purple-r01' 확인 → Next step. (purple-r01~r15 를 모두 포함)

인덱스 패턴 만들기 — ④ 시간 필드 timestamp → 만들기



Time field 에 timestamp 선택 → Create index pattern. 끝. 이제 Discover·Visualize 에서 purple-* 를 고를 수 있다

데이터가 안 보이면 ? — 십중팔구 ' 시간 범위 '

Discover·Visualize 는 기본이 ' 최근 15 분 ' 이다 . 이 실습 데이터는 90 일치 (과거) 라 , 기본 범위에서는 아무것도 안 보인다 .

- 오른쪽 위 달력에서 Last 90 days (또는 2026-08-01 ~ 2026-11-02) 로 바꾼다 .

- Dev Tools 쿼리에는 시간 선택기가 없다 — 쿼리 안의 range 로 직접 기간을 건다 (예 : "gte": "now-90d").

체크리스트

① gen 으로 적재했나 ② 인덱스 패턴 purple-* 만들었나 ③ 시간 범위 90 일로 바꿨나

이 세 가지만 맞으면 다음 장부터의 모든 예제가 그대로 돌아간다 .

집계는 두 곳에서 한다 — Dev Tools 와 Visualize

Dev Tools	쿼리 (DSL) 를 글로 직접 쓴다 . 중첩 · 조건 · 비율까지 다 된다 . 이 과목의 ' 정식 ' 도구
Visualize	마우스로 버킷 · 지표를 고른다 . 막대 · 표 · 그래프가 바로 나온다 . 같은 집계를 ' 그림 ' 으로

둘은 같은 집계를 다른 방법으로 부른다 . 수업에서는 Dev Tools 로 DSL 을 읽고 → ask 로 정답 키와 맞춰 보고 → Visualize 로 같은 그림을 그린다 .

실제 공격은 하지 않는다 . 생성기가 정상 99.9% 에 공격 로그를 섞어 인덱서에 적재한 것을 집계로 찾는다 . 외부 IP 는 전부 가상이다 .

해부

집계 쿼리의 뼈대 — Dev Tools 요청 (DSL)

```
POST purple-r01/_search
{
  "size": 0,
  "query": {
    "term": {
      "decoder.name": "web-accesslog"
    }
  },
  "aggs": {
    "by_vhost": {
      "terms": {
        "field": "data.vhost",
        "size": 5
      }
    }
  }
}
```

Dev Tools 요청 편집기에 이 DSL 을 치고 ▶ . size:0 = 원문 로그는 안 가져오고 집계 숫자만 . query= 먼저 거를 조건 , aggs= 묶기 , terms= 값별 버킷 , size= 상위 몇 개 . 결과 (각 버킷 doc_count) 는 다음 장 Visualize 로

해부

집계 쿼리의 네 부분 — 외우면 끝

size: 0	원문 로그는 필요 없다, 집계 숫자만 달라. (안 쓰면 로그 10건이 딸려 온다)
query	집계하기 전에 먼저 거를 조건 — 예: 웹 로그만, 404 만. 범위를 좁힐수록 빠르고 정확
aggs	집계를 적는 곳. 이름을 내 맘대로 짓는다 (by_ip· 고유 URL …)
버킷 vs 지표	버킷 (bucket)= 묶음을 나눈다 (terms·date_histogram). 지표 (metric)= 각 묶음을 숫자로 잰다 (count·cardinality·sum)

거의 모든 집계 = query 로 거르고 → 버킷으로 나누고 → 각 버킷을 지표로 잰다. 이 세 줄이 전부다.

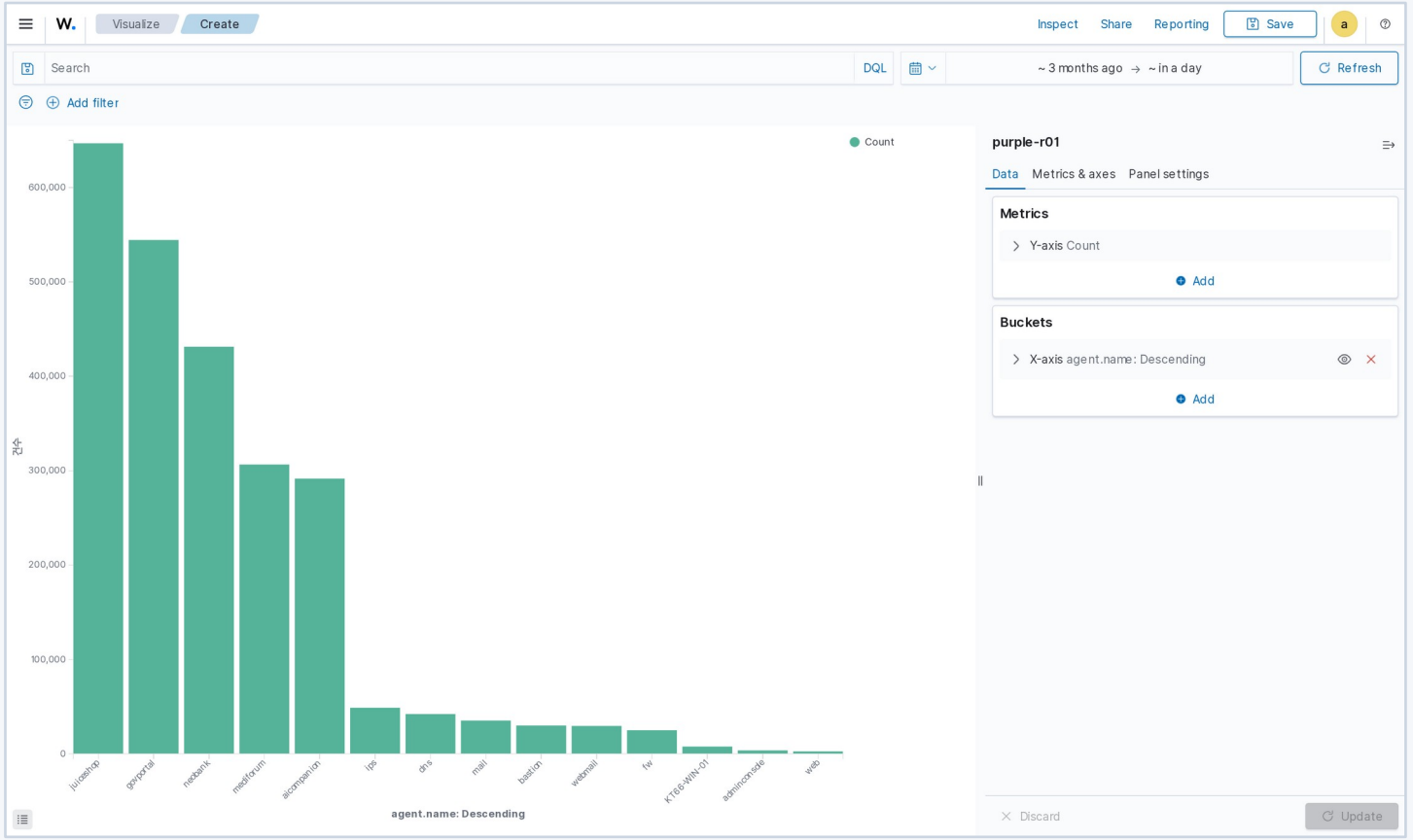
① Terms — 값별로 묶어 센다 (Dev Tools)

```
POST purple-r01/_search
{
  "size": 0,
  "query": {
    "term": {
      "decoder.name": "web-accesslog"
    }
  },
  "aggs": {
    "by_vhost": {
      "terms": {
        "field": "data.vhost",
        "size": 5
      }
    }
  }
}
```

terms: data.vhost 의 값마다 버킷을 만들고 doc_count(건수) 로 센다 . size:5 = 상위 5 개만 . ' 누가 가장 많이 ' 의 기본

Terms(버킷)

① Terms — Visualize 로 같은 집계 (막대그래프)



오른쪽 패널 : Buckets 에 X 축 Terms agent.name, Metrics 에 Y 축 Count. juiceshop 이 가장 많이 말한다 = 정상의 크기 . 묶음 하나가 막대 하나

② 지표 (Metric) — 각 묶음에서 '무엇을 잴까'

버킷으로 묶었으면, 각 묶음을 숫자로 잰다. 기본은 건수 (count) 지만 다른 것도 잴 수 있다.

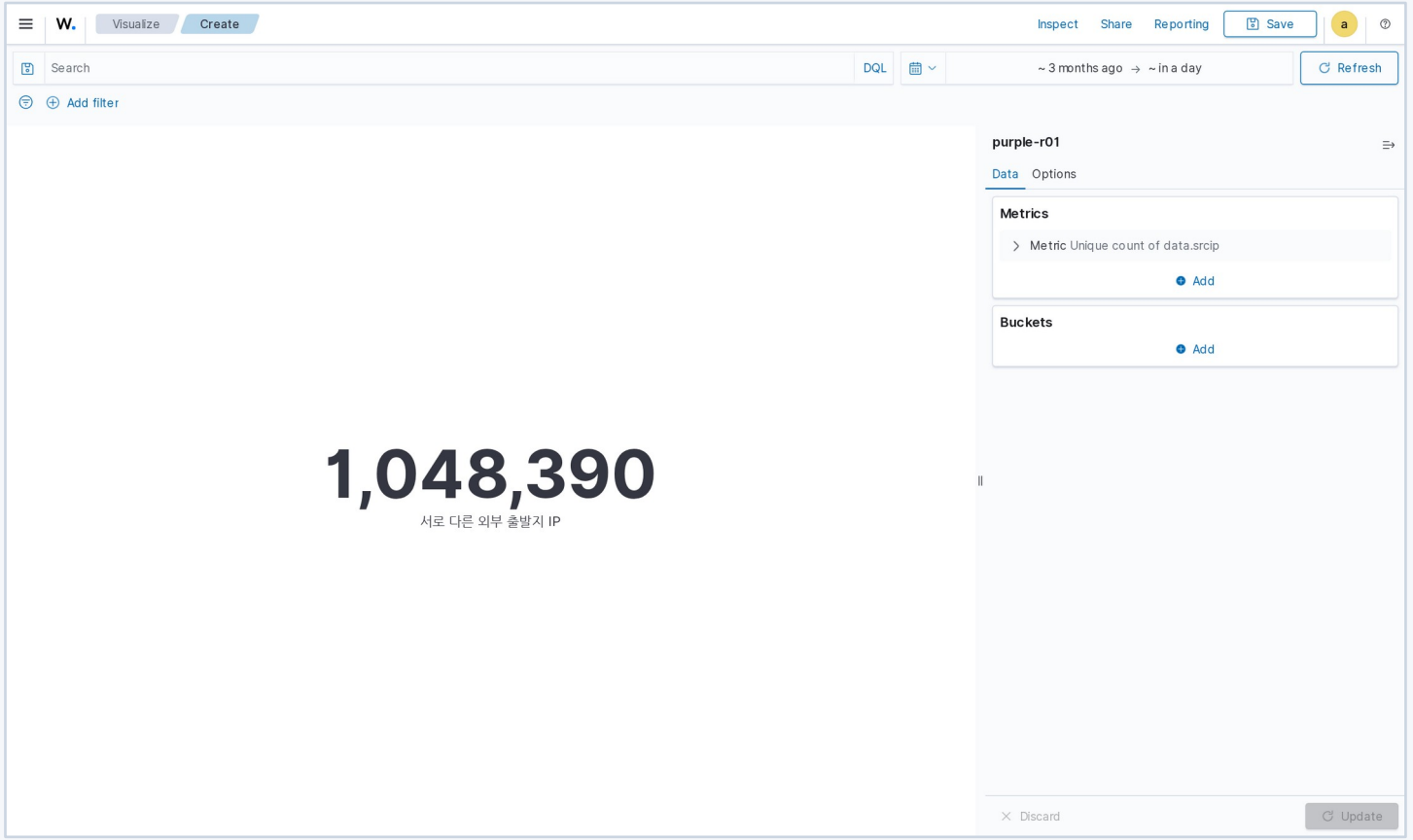
지표	무엇을 재나	어디에 쓰나
Count	묶음 안의 건수	기본 — 몇 번 왔나
Cardinality	서로 다른 값의 개수 (고유 수)	얼마나 넓게 훑었나
Sum	숫자 필드의 합계	업로드 바이트 총량 (유출)
Max / Min	최대 · 최소	가장 큰 응답, 처음 · 마지막 시각
Average	평균	평균 프롬프트 길이 (AI)

③ Cardinality — 서로 다른 값이 몇 개 (Dev Tools)

```
POST purple-r01/_search
{
  "size": 0,
  "query": {
    "term": {
      "decoder.name": "web-accesslog"
    }
  },
  "aggs": {
    "고유_방문IP": {
      "cardinality": {
        "field": "data.srcip"
      }
    }
  }
}
```

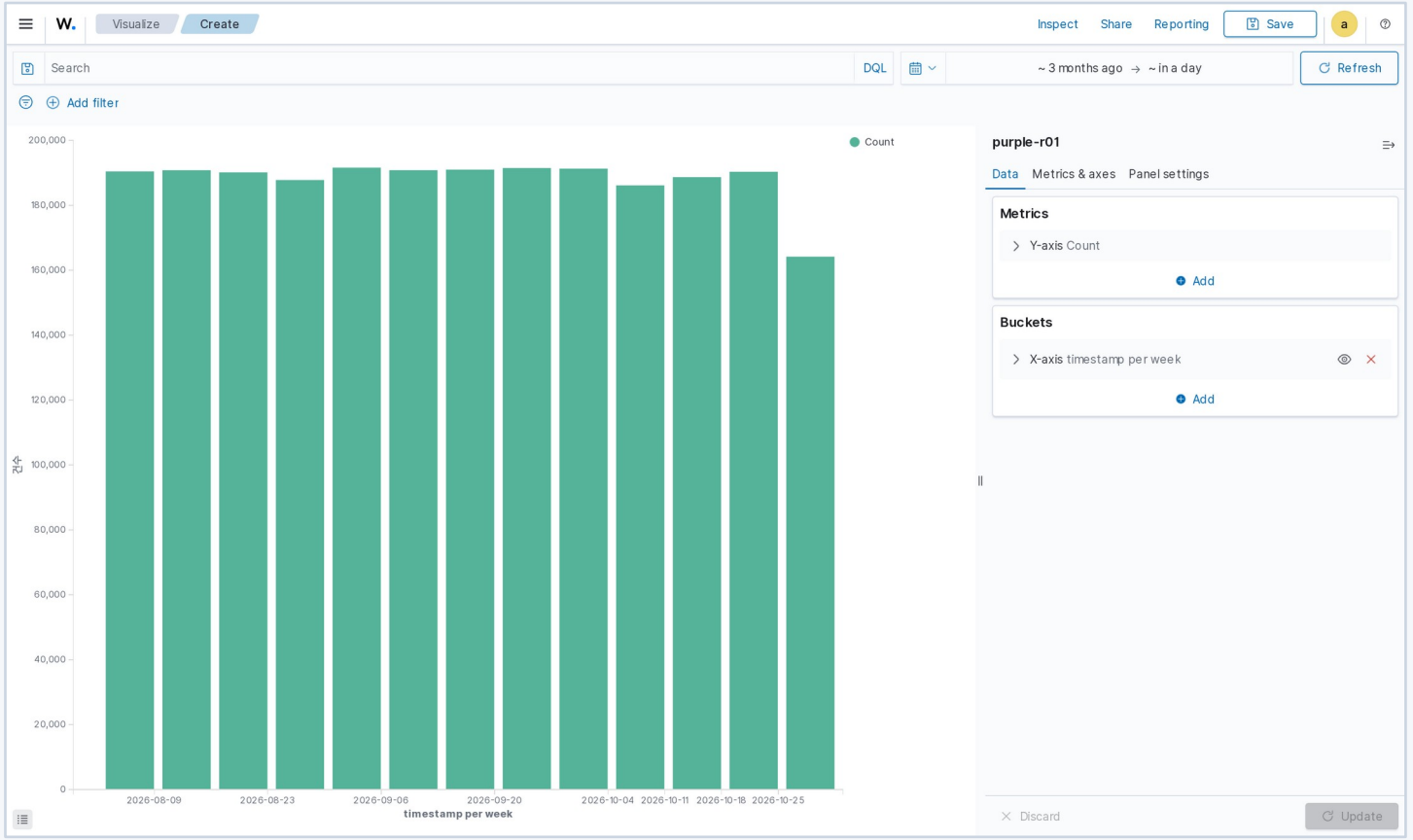
cardinality: data.srcip 의 '서로 다른' 개수 . 건수가 아니라 종류 수다 . '많이 온 IP' 가 아니라 '넓게 훑은 IP' 를 가르는 핵심 지표

③ Cardinality — Visualize 지표 한 개 (큰 숫자)



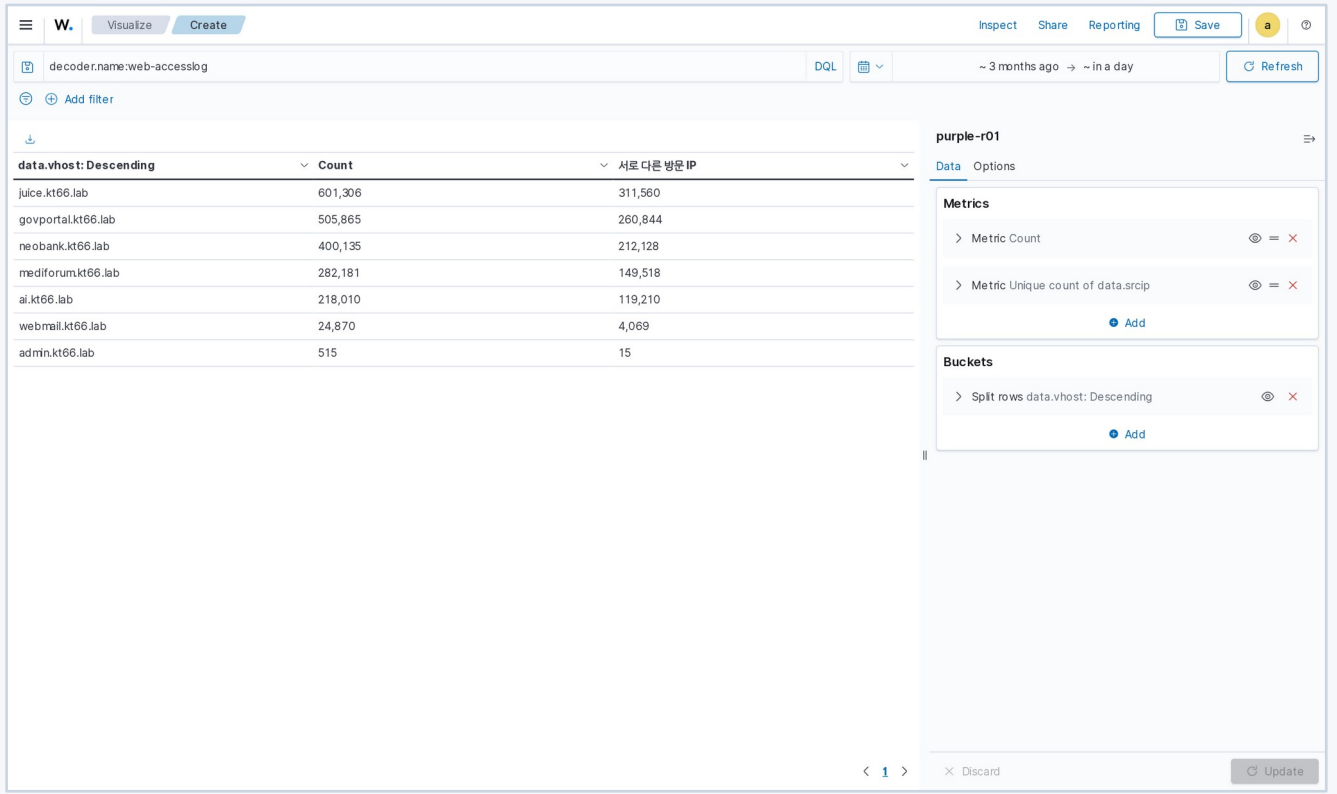
Metric 시각화 : Unique Count of data.srcip = 1,048,390. 90 일간 서로 다른 외부 출발지 IP 수 . (고유 수는 인덱서에서 근사치라 정답 키와 약간 다를 수 있다)

④ Date Histogram — 시간을 묶는다 (Visualize)



X 축을 timestamp 의 7 일 (주) 간격으로 묶었다 . 90 일 전체가 한 화면에 — 계절성 · 공백 · 급증이 바로 보인다 . 간격을 분 · 시간 · 일로 바꾸면 보는 해상도가 바뀐다

⑤ 버킷 속 버킷 · 지표 — 중첩 집계 (Visualize 데이터 표)



Split rows Terms data.vhost(버킷) + Metric Unique Count data.srcip(지표). vhost 별 요청 수와 ' 서로 다른 방문 IP 수 ' 를 한 표에 . 요청 수 ÷ 방문 IP = 서비스의 성격

⑥ Filters — 조건별로 한 줄에 (Dev Tools)

```
POST purple-r01/_search
{
  "size": 0,
  "query": {
    "term": {
      "decoder.name": "web-accesslog"
    }
  },
  "aggs": {
    "응답코드별": {
      "filters": {
        "filters": {
          "정상_200": {
            "term": {
              "data.id": "200"
            }
          },
          "없음_404": {
            "term": {
              "data.id": "404"
            }
          },
          "차단_403": {
            "term": {
              "data.id": "403"
            }
          }
        }
      }
    }
  }
}
```

filters: 200·404·403 각각의 건수를 한 번에 . 같은 집계 안에서 '막힌 것 vs 통과한 것'을 나란히 센다 . R03(차단 → 통과) 의 핵심

⑦ bucket_selector — 조건을 넘는 묶음만 (Dev Tools)

```
POST purple-r01/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "decoder.name": "web-accesslog"
          }
        },
        {
          "term": {
            "data.id": "404"
          }
        }
      ]
    }
  },
  "aggs": {
    "by_ip": {
      "terms": {
        "field": "data.srcip",
        "size": 5,
        "order": {
          "고유URL": "desc"
        }
      },
      "aggs": {
        "고유URL": {
          "cardinality": {
            "field": "data.url"
          }
        },
        "많이_훑은곳": {
          "bucket_selector": {
            "buckets_path": {
              "u": "고유URL"
            },
            "script": "params.u >= 50"
          }
        }
      }
    }
  }
}
```

먼저 404 만 거르고 → IP 별 고유 URL 수 (cardinality) 를 재고 → bucket_selector 로 '고유 URL 50 개 이상' 인 IP 만 남긴다. 이게 바로 경보 (모니터) 의 조건이 된다

⑦ bucket_selector = 경보의 조건 → 모니터

집계 결과 중 '기준을 넘는 묶음'만 남기는 것이 bucket_selector 다. 이 조건이 OpenSearch Alerting 모니터의 트리거가 된다.

예 (R02)	404 받은 서로 다른 URL 이 100 개 이상인 IP — 남으면 경보
트리거	`ctx.results[0].aggregations.by_ip.buckets.size() > 0` (남은 버킷이 있으면 울린다)

경보는 '한 줄 서명'이 아니라 '조건을 넘는 집계 결과'로 운다. 분산·장기 공격은 한 줄로는 안 잡히고, 전체 기간을 센 집계에서만 드러난다.

축 뒤집기 — 분산 공격을 드러내는 한 수

정찰·대입을 'IP 별'로 세면 각 IP는 한두 번뿐이라 안 보인다. 축을 '경로별' 또는 '계정별'로 뒤집으면, 같은 목표를 두드린 '서로 다른 IP 수'가 튄다.

축(버킷)	지표	드러나는 것
IP 별	요청 수	대량 사용자(정상) — 분산 공격은 안 보임
경로(URL) 별	서로 다른 IP 수	숨은 API 한 곳을 IP 226곳이 두드림 (R02)
계정 별	서로 다른 실패 IP 수	한 계정에 1,200 IP 분산 대입 (R06)

'누가 많이 왔나(Terms+Count)'로 안 보이면, '무엇을 여러 곳이 두드렸나(Terms+Cardinality, 축을 바꿔)'로 본다. 이 뒤집기가 분산 공격 탐지의 핵심이다.

Visualize 따라하기 — 막대그래프 만들기

1	☰ 메뉴 → Visualize → Create visualization → Vertical Bar
2	인덱스 패턴 purple-* 선택
3	Buckets → Add → X-axis → Aggregation: Terms → Field: agent.name → Size 15
4	Metrics 는 Y-axis Count 그대로 (건수)
5	오른쪽 아래 Update → 막대그래프 완성

데이터 표가 필요하면 Create → Data Table. 서로 다른 수가 필요하면 Metrics 에서 Unique Count 를 고른다. 시간 추이는 X-axis 를 Date Histogram 으로.

결과를 읽는 법 · 정답 키와 비교

- 기준일을 2026-11-01 로 고정하면 숫자가 동봉 정답 키와 같다 (다른 날짜면 분포는 같고 시각만 밀린다).
- 고유 수 (cardinality) 는 인덱서에서 ' 근사치 ' 다 — 정답 키 (정확) 와 1~2% 차이는 정상 .
- doc_count(건수) 는 정확히 일치한다 .
- 시간은 KST(+09:00) 로 묶는다 — time_zone 을 안 주면 UTC 라 하루가 밀린다 .

미션 흐름 : DSL 읽기 → curl 로 직접 질의 → ask 로 정답 키와 비교 → Visualize 로 같은 그림 . 네 가지가 한 세트다 .

세 겹의 탐지 — 각자 기억하는 시간이 다르다

같은 공격도 보는 '층'에 따라 다르게 잡힌다. 이 과목은 맨 아래 '집계' 층을 다룬다.

층	기억하는 시간	강한 것
서명 (Suricata·ModSec)	한 줄 · 초	우회 패턴, 업로드, 속도
상관 (Wazuh 규칙)	분 ~ 일	차단→통과, 계정 확산
집계 · 모니터 (OpenSearch)	전체 기간	분산 대입, 장기 비콘, 석 달 체인

분산 · 장기 공격은 위 두 층에서 '0 건'이 정상이다 — 그래서 집계가 먼저다. 각 라운드에서 서명 · 상관이 몇 건 울렸는지를 집계와 나란히 놓고 비교한다.

정리 — 이 여섯 개면 13 개 라운드를 푼다

Terms	값별로 묶어 센다 — 누가 많이
Cardinality	서로 다른 수 — 얼마나 넓게
Date Histogram	시간으로 묶는다 — 언제 · 규칙성
Filters	조건별 건수 — 막힘 vs 통과
Sum·Max·Avg	크기를 잰다 — 유출 · 응답
bucket_selector	조건 넘는 것만 — 경보

다음 모듈부터 13 개 라운드에서 이 여섯 개를 섞어 쓰며, 정찰 · 웹공격 · 지속화 · 권한상승 · 자격증명 · 횡적이동 ·C2· 유출 ·AI 공격을 집계로 찾는다.